

Best Programming Practice

1. Use meaningful class names (e.g., `Student`, `Employee`) and method names (e.g., `displayDetails`).
2. Encapsulate data using `private` fields and provide `getter` and `setter` methods.
3. Follow proper naming conventions (`camelCase` for attributes and methods).
4. Always provide constructors to initialize class attributes.
5. Use comments for clarity and better readability.

Sample Program 1: Food Delivery App

Real-World Analogy

Imagine a **food delivery app** like Swiggy or Uber Eats. The app deals with **restaurants**, and each restaurant has specific details like its name, location, and the food items it serves.

Step 1: Define the Class

The `Restaurant` class represents the blueprint for creating restaurant objects.

// Class Definition

```
public class Restaurant {  
    // Fields (Attributes)  
    private String name;  
    private String location;  
    private String[] foodItems;
```

// Constructor

```
public Restaurant(String name, String location, String[] foodItems) {  
    this.name = name;  
    this.location = location;  
    this.foodItems = foodItems;  
}
```

// Method to display restaurant details

```
public void displayDetails() {  
    System.out.println("Restaurant Name: " + name);  
    System.out.println("Location: " + location);  
    System.out.println("Food Items: ");  
    for (String item : foodItems) {
```

```

        System.out.println("- " + item);
    }
}

// Method to check if a food item is available
public boolean isFoodAvailable(String food) {
    for (String item : foodItems) {
        if (item.equalsIgnoreCase(food)) {
            return true;
        }
    }
    return false;
}
}

```

Step 2: Create Objects from the Class

Use the class to create specific restaurant objects.

```

// Main Class to Test
public class Main {
    public static void main(String[] args) {
        // Define food items for restaurants
        String[] foodItems1 = {"Pizza", "Pasta", "Burger"};
        String[] foodItems2 = {"Sushi", "Ramen", "Tempura"};

        // Create Restaurant objects
        Restaurant restaurant1 = new Restaurant("Italian Delight", "Downtown", foodItems1);
        Restaurant restaurant2 = new Restaurant("Tokyo Treats", "Uptown", foodItems2);

        // Display details of each restaurant
        System.out.println("=== Restaurant 1 ===");
        restaurant1.displayDetails();
        System.out.println("\n=== Restaurant 2 ===");
        restaurant2.displayDetails();

        // Check food availability
        System.out.println("\nChecking Food Availability:");
        System.out.println("Is Pasta available in Italian Delight? " +
            restaurant1.isFoodAvailable("Pasta"));
    }
}

```

```
        System.out.println("Is Sushi available in Italian Delight? " +  
restaurant1.isFoodAvailable("Sushi"));  
    }  
}
```

Step 3: Output

=== Restaurant 1 ===

Restaurant Name: Italian Delight

Location: Downtown

Food Items:

- Pizza
- Pasta
- Burger

=== Restaurant 2 ===

Restaurant Name: Tokyo Treats

Location: Uptown

Food Items:

- Sushi
- Ramen
- Tempura

Checking Food Availability:

Is Pasta available in Italian Delight? true

Is Sushi available in Italian Delight? false

In-depth explanation of Key Aspects

1. Fields (Attributes)

- Fields store the data for the class.
- Example: `name`, `location`, and `foodItems` represent the state of a restaurant.

2. Constructor

- A constructor initializes the fields when an object is created.
- Example: The `Restaurant` constructor sets `name`, `location`, and `foodItems`.

3. Methods

- Methods define the behavior of the objects.
- Example:
 - `displayDetails()`: Displays the details of a restaurant.

- `isFoodAvailable(String food)`: Checks if a specific food item is available.

4. Encapsulation

- The fields are marked as `private` and accessed using methods to ensure controlled data access and modification.

5. Object Creation

- Objects are created using the `new` keyword.

Example:

```
Restaurant restaurant1 = new Restaurant("Italian Delight", "Downtown", foodItems1);
```

6. Memory Allocation

- Each object has its own memory space for attributes but shares methods.

Level 1 Practice Programs

1. Program to Display Employee Details

Problem Statement: Write a program to create an **Employee** class with attributes **name**, **id**, and **salary**. Add a method to display the details.

```
import java.util.Scanner;
class Employee {
    private String name;
    private int id;
    private double salary;

    //making constructor
    public Employee() {

        //creating scanner object
        Scanner input = new Scanner(System.in);

        //taking user inputs for name, id and salary
        System.out.print("Enter name : ");
        name = input.nextLine();
        System.out.print("Enter ID : ");
        id = input.nextInt();
        System.out.print("Enter salary : ");
        salary = input.nextInt();

    }

    //method to print employee details
    public void printDetails() {
        System.out.println("Name : " + name);
        System.out.println("Id : " + id);
        System.out.println("Salary : " + salary);
    }
}

public class EmployeeDetails {
    public static void main(String[] args) {

        //creating scanner object
```

```

Scanner input = new Scanner(System.in);

//creating a variable numberOfEmployees to indicate number of
employees
System.out.print("Enter number of employees : ");
int numberOfEmployees = input.nextInt();

//creating an empArray of object to store details of employees
Employee[] empArray = new Employee[numberOfEmployees];

//printing details of employees
for(int i=0; i<numberOfEmployees; i++) {
    System.out.println("\nEnter details of employee " + i+1 +
" : ");
    empArray[i] = new Employee();
}

//printing details of employees
for(int i=0; i<numberOfEmployees; i++) {
    System.out.println("\nDetails of employee " + i+1 + " :
");
    empArray[i].printDetails();
}
//closing input
input.close();
}
}

```

2. Program to Compute Area of a Circle

Problem Statement: Write a program to create a **Circle** class with an attribute **radius**. Add methods to calculate and display the area and circumference of the circle.\

```

import java.util.Scanner;
class Circle {

```

```
private final double PI = 3.14;
private double radius;
private double area;
private double circumference;

//making constructor
public Circle() {

    //creating scanner object
    Scanner input = new Scanner(System.in);

    //taking user input for radius
    System.out.print("Enter radius : ");
    radius = input.nextDouble();

}

//method to calculate area and circumference
public void calculateAreaAndCircumference() {

    //calculating area
    area = PI * radius * radius;

    //calculating circumference
    circumference = 2 * PI * radius;

}

//method to print area and circumference
public void printAreaAndCircumference() {
    System.out.println("Area : " + area);
    System.out.println("Circumference : " + circumference);
}
}

public class CircleDetails {
    public static void main(String[] args) {

        //creating scanner object
        Scanner input = new Scanner(System.in);

        //creating object of circle class
```

```
Circle cir = new Circle();

//calculating area and circumference
cir.calculateAreaAndCircumference();

//printing area and circumference of circle
cir.printAreaAndCircumference();

//closing input
input.close();
    }
}
```

3. Program to Handle Book Details

Problem Statement: Write a program to create a **Book** class with attributes **title**, **author**, and **price**. Add a method to display the book details.

```
import java.util.Scanner;
class Book {
    private String title;
    private String author;
    private double price;

    //making constructor
    public Book() {

        //creating scanner object
        Scanner input = new Scanner(System.in);

        //taking user inputs for title, author and price of book
        System.out.print("Enter title of book : ");
        title = input.nextLine();
        System.out.print("Enter author's name : ");
        author = input.nextLine();
        System.out.print("Enter price of book : ");
        price = input.nextDouble();

    }
}
```



```
//method to print book details
public void printDetails() {
    System.out.println("Title : " + title);
    System.out.println("Author : " + author);
    System.out.println("Price : " + price);
}
}
public class BookDetails {
    public static void main(String[] args) {

        //creating scanner object
        Scanner input = new Scanner(System.in);

        //creating a variable numberOfBooks to indicate number of books
        System.out.print("Enter number of books : ");
        int numberOfBooks = input.nextInt();

        //creating an book Array of objects to store details of books
        Book[] bookArray = new Book[numberOfBooks];

        //taking details of books as user input
        for(int i=0; i<numberOfBooks; i++) {
            System.out.println("\nEnter details of book " + (i+1) + "
: ");

            bookArray[i] = new Book();
        }

        //printing details of books
        for(int i=0; i<numberOfBooks; i++) {
            System.out.println("\nDetails of book " + (i+1) + " : ");
            bookArray[i].printDetails();
        }
        //closing input
        input.close();
    }
}
```

4. Program to Track Inventory of Items

Problem Statement: Create an `Item` class with attributes `itemCode`, `itemName`, and `price`. Add a method to display item details and calculate the total cost for a given quantity.

```
import java.util.Scanner;
class Item {
    private String itemCode;
    private String itemName;
    private double price;

    //making constructor
    public Item(String itemCode, String itemName, double price) {
        this.itemCode = itemCode;
        this.itemName = itemName;
        this.price = price;
    }

    //method to calculate cost of x(in kg) quantity
    public double calcCost(double x) {
        //calculating the cost
        double cost = x*(this.price);

        return cost;
    }

    //method to print item details
    public void printDetails(double quantity) {
        System.out.println("\n----Item details----");
        System.out.println("Item Code : " + itemCode);
        System.out.println("Item name : " + itemName);
        System.out.println("Price : " + price);
        System.out.println("Cost of " + quantity + " kg is " +
this.calcCost(quantity));
    }
}
public class ItemDetails {
    public static void main(String[] args) {

        //creating scanner object
```

```

Scanner input = new Scanner(System.in);

//taking user inputs for code, name and price of item
System.out.print("Enter item code : ");
String itemCode = input.nextLine();
System.out.print("Enter item name : ");
String itemName = input.nextLine();
System.out.print("Enter price of item (per unit) : ");
Double price = input.nextDouble();

//taking user input for quantity
System.out.print("Enter quantity of item (in kg) : ");
Double quantity = input.nextDouble();
//creating a object of Item class
Item item = new Item(itemCode, itemName, price);

//printing details of item
item.printDetails(quantity);

//closing input
input.close();
}
}

```

5. Program to Handle Mobile Phone Details

Problem Statement: Create a **MobilePhone** class with attributes **brand**, **model**, and **price**. Add a **method** to display all the details of the phone. The **MobilePhone** class uses attributes to store the phone's characteristics. The **method** is used to retrieve and display this information for each **object**.

```

import java.util.Scanner;
class MobilePhone {
    private String brand;
    private String model;
    private double price;

```

```
//making constructor
public MobilePhone(String brand, String model, double price) {
    this.brand = brand;
    this.model = model;
    this.price = price;
}

//method to print mobile details
public void printDetails() {
    System.out.println("\n---Mobile details---");
    System.out.println("Brand : " + brand);
    System.out.println("Model : " + model);
    System.out.println("Price : " + price);
}
}

public class MobileDetails {
    public static void main(String[] args) {

        //creating scanner object
        Scanner input = new Scanner(System.in);

        //taking user inputs for brand, model and price
        System.out.print("Enter brand : ");
        String brand = input.nextLine();
        System.out.print("Enter model : ");
        String model = input.nextLine();
        System.out.print("Enter price of mobile : ");
        Double price = input.nextDouble();

        //creating a object of MobilePhone class
        MobilePhone mobile = new MobilePhone(brand, model, price);

        //printing details of mobile
        mobile.printDetails();

        //closing input
        input.close();
    }
}
```

Level 2 Practice Programs

1. Program to Simulate Student Report

Problem Statement: Create a `Student` class with attributes `name`, `rollNumber`, and `marks`. Add two methods:

- To calculate the grade based on the marks.
- To display the student's details and grade.

Explanation: The `Student` class organizes all relevant details about a student as attributes. Methods are used to calculate the grade and provide a way to display all information.

```
import java.util.Scanner;
class Student {
    private String name;
    private int rollNo;
    private double marks;
    private char grade;

    //making constructor
    public Student(String name, int rollNo, double marks) {
        this.name = name;
        this.rollNo = rollNo;
        this.marks = marks;
    }

    //method to assigning grades of student
    public void assignGrade(double marks) {
        //assigning grade
        if(marks>90) {
            this.grade = 'A';
        } else if(marks>75) {
            this.grade = 'B';
        } else if(marks>50) {
            this.grade = 'C';
        } else if(marks>33) {
            this.grade = 'D';
        } else {
            this.grade = 'F';
        }
    }
}
```

```
    }  
}  
//method to print Student details  
public void printDetails() {  
  
    // calling the assign grade method to assign grade  
    this.assignGrade(grade);  
    System.out.println("\n---student details---");  
    System.out.println("Name : " + name);  
    System.out.println("Roll number : " + rollNo);  
    System.out.println("Marks : " + marks);  
    System.out.println("Grade : " + grade);  
}  
}  
public class StudentDetails {  
    public static void main(String[] args) {  
  
        //creating scanner object  
        Scanner input = new Scanner(System.in);  
  
        //taking user inputs for name, rollNo and marks  
        System.out.print("Enter name : ");  
        String name = input.nextLine();  
        System.out.print("Enter roll number : ");  
        int rollNo = input.nextInt();  
        System.out.print("Enter marks : ");  
        double marks = input.nextDouble();  
  
        //creating object of student class  
        Student student = new Student(name, rollNo, marks);  
  
        //printing details of student  
        student.printDetails();  
  
        //closing input  
        input.close();  
    }  
}
```

2. Program to Simulate an ATM

Problem Statement: Create a `BankAccount` class with attributes `accountHolder`, `accountNumber`, and `balance`. Add methods for:

- Depositing money.
- Withdrawing money (only if sufficient balance exists).
- Displaying the current balance.

Explanation: The `BankAccount` class stores bank account details as attributes. The methods allow interaction with these attributes to modify and view the account's state.

```
import java.util.Scanner;
class BankAccount {
    private String accountHolder;
    private String accountNumber;
    private double balance = 0;

    //method to deposit money
    public void deposit(double amount) {
        //increasing the balance with amount
        this.balance += amount;
    }
    //method to open a account (constructor)
    public BankAccount(String accountHolder, String accountNumber) {
        this.accountHolder = accountHolder;
        this.accountNumber = accountNumber;
    }
    //method to withdraw money
    public void withdraw(double amount) {
        //checking if amount is valid and balance is sufficient to withdraw
        if(amount >= 0 && this.balance >= amount) {
            //decreasing withdraw amount from balance
            this.balance -= amount;
        } else {
            System.out.println("Insufficient balance..");
        }
    }
    //method to display current balance
    public void displayBalance() {
        System.out.println("your current balance is : " + this.balance);
    }
}
```

```
public class ATM {
    public static void main(String[] args) {
        //creating scanner object
        Scanner input = new Scanner(System.in);

        //taking user input for account holder and account number
        System.out.print("Enter your name : ");
        String accountHolder = input.nextLine();
        System.out.print("Enter your account number : ");
        String accountNumber = input.nextLine();

        //creating object
        BankAccount account = new BankAccount(accountHolder, accountNumber);

        while(true) {
            System.out.println("press
\n1----->deposit\n2----->withdraw\n3----->check balance");

            //creating a variable choice to indicate users choice for operations
            int choice = input.nextInt();

            if(choice == 1) {
                System.out.print("Enter amount : ");
                account.deposit(input.nextDouble());
            } else if (choice == 2) {
                System.out.print("Enter amount : ");
                account.withdraw(input.nextDouble());
            } else if (choice == 3) {
                account.displayBalance();
            } else {
                System.out.println("Invalid choice...");
            }
            //exiting if user want else continuing
            System.out.println("Enter 0 to exit else press any key ....");
            if(input.nextInt() == 0) {
                break;
            }
        }

        //closing the input
        input.close();
    }
}
```


3. Program to Check Palindrome String

Problem Statement: Create a `PalindromeChecker` class with an attribute `text`. Add **methods** to:

- Check if the `text` is a palindrome.
- Display the result.

Explanation: The `PalindromeChecker` class holds the `text` attribute. The **methods** operate on this attribute to verify its palindrome status and display the result.

```
import java.util.Scanner;

class PalindromeChecker {

    //creating variable to store text

    private String text;

    //constructor of class to initialize text

    public PalindromeChecker(String text) {

        this.text = text;

    }

    // Method to check if a string is palindrome by comparing start and end
    characters

    public void isPalindrome() {

        //creating variables start and end to indicate starting and ending index
        of text

        int start = 0;

        int end = this.text.length() - 1;

        //creating variable flag

        boolean flag = true;
```

```
while (start < end) {

    if (this.text.charAt(start) != this.text.charAt(end)) {

        System.out.println("text " + this.text + " is not palindrome");

        //initializing flag to false because text is not palindrome

        flag = false;

        break;

    }

    start++;

    end--;

}

//if flag == true then text is palindrome

if(flag) {

    System.out.println("text " + this.text + " is palindrome");

}

}

public static void main(String[] args) {

    //creating scanner object

    Scanner input = new Scanner(System.in);

    //creating variable text to indicate text and taking user input for text
```

```
System.out.print("Enter text : ");

String text = input.nextLine();

//creating object

PalindromeChecker object = new PalindromeChecker(text);

//calling the method isPalindrome to check whether the text is
palindrome or not

object.isPalindrome();

//closing the input

input.close();

}

}
```

4. Program to Model a Movie Ticket Booking System

Problem Statement: Create a `MovieTicket` class with attributes `movieName`, `seatNumber`, and `price`. Add **methods** to:

- Book a ticket (assign seat and update `price`).
- Display ticket details.

Explanation: The `MovieTicket` class organizes ticket information with attributes. The **methods** handle booking logic and display ticket details.

5. Program to Simulate a Shopping Cart

Problem Statement: Create a `CartItem` class with attributes `itemName`, `price`, and `quantity`. Add **methods** to:

- Add an item to the cart.
- Remove an item from the cart.
- Display the total cost.

Explanation: The `CartItem` class models a shopping cart item. The **methods** handle cart operations like adding or removing items and calculating the total cost.

```
import java.util.Scanner;
class CartItem {
    String itemName;
    double price;
    int quantity;

    // Constructor
    CartItem(String itemName, double price, int quantity) {
        this.itemName = itemName;
        this.price = price;
        this.quantity = quantity;
    }

    // Displaying the details of the cart item
    void displayItem() {
        System.out.println(" Item name : " + itemName + "\n Price: " + price + "\n
Quantity: " + quantity);
    }
}

public class ShoppingCart {
    //creating Array for storing cart items
    CartItem[] cart = new CartItem[10];
    //creating variable cartSize to indicate Current number of items in the cart
    int cartSize = 0;

    // Adding an item to the cart
    void addItem(String itemName, double price, int quantity) {
        if (cartSize < cart.length) {
            cart[cartSize++] = new CartItem(itemName, price, quantity);
            System.out.println(itemName + " added to the cart.");
        } else {
            System.out.println("Cart is full!");
        }
    }
}
```

```
// Removing an item by its name
void removeItemByName(String itemName) {
    int indexToRemove = -1;

    // Finding the index of the item with the given name
    for (int i = 0; i < cartSize; i++) {
        if (cart[i].itemName.equalsIgnoreCase(itemName)) {
            indexToRemove = i;
            break;
        }
    }

    if (indexToRemove == -1) {
        System.out.println("Item \"" + itemName + "\" not found in the cart.");
        return;
    }

    // Shifting elements to remove the item at the found index
    for (int i = indexToRemove; i < cartSize - 1; i++) {
        cart[i] = cart[i + 1];
    }

    // Reducing the size and set the last item to null
    cart[--cartSize] = null;
    System.out.println("Item \"" + itemName + "\" removed from the cart.");
}

// Displaying all items in the cart
void displayCart() {
    if (cartSize == 0) {
        System.out.println("The cart is empty.");
        return;
    }

    System.out.println("Cart Contents:");
    for (int i = 0; i < cartSize; i++) {
        System.out.print(i + ": ");
        cart[i].displayItem();
    }
}

// Displaying the total cost
void displayTotalCost() {
    double total = 0;
    for (int i = 0; i < cartSize; i++) {
        total += cart[i].price * cart[i].quantity;
    }
}
```

```
}
System.out.println("Total Cost: $" + total);
}

public static void main(String[] args) {

    //creating scanner object
    Scanner input = new Scanner(System.in);

    //creating object of shopping cart
    ShoppingCart myCart = new ShoppingCart();

    while (true) {
        System.out.println("\nChoose an option:");
        System.out.println("1. Add Item to Cart");
        System.out.println("2. Remove Item from Cart");
        System.out.println("3. Display Cart");
        System.out.println("4. Display Total Cost");
        System.out.println("5. Exit");
        System.out.print("Enter your choice: ");
        int choice = input.nextInt();
        // Consuming newline
        input.nextLine();

        switch (choice) {
            case 1:
                System.out.print("Enter item name: ");
                String itemName = input.nextLine();
                System.out.print("Enter item price: ");
                double price = input.nextDouble();
                System.out.print("Enter item quantity: ");
                int quantity = input.nextInt();
                myCart.addItem(itemName, price, quantity);
                break;

            case 2:
                System.out.print("Enter the name of the item to remove: ");
                String removeName = input.nextLine();
                myCart.removeItemByName(removeName);
                break;

            case 3:
                myCart.displayCart();
                break;
```

```
        case 4:
            myCart.displayTotalCost();
            break;

        case 5:
            System.out.println("Exiting program. Thank you!");
            input.close();
            return;

        default:
            System.out.println("Invalid choice! Please try again.");
    }

}

}
```